24小时快速入门大热语言Rust！学不会来找我

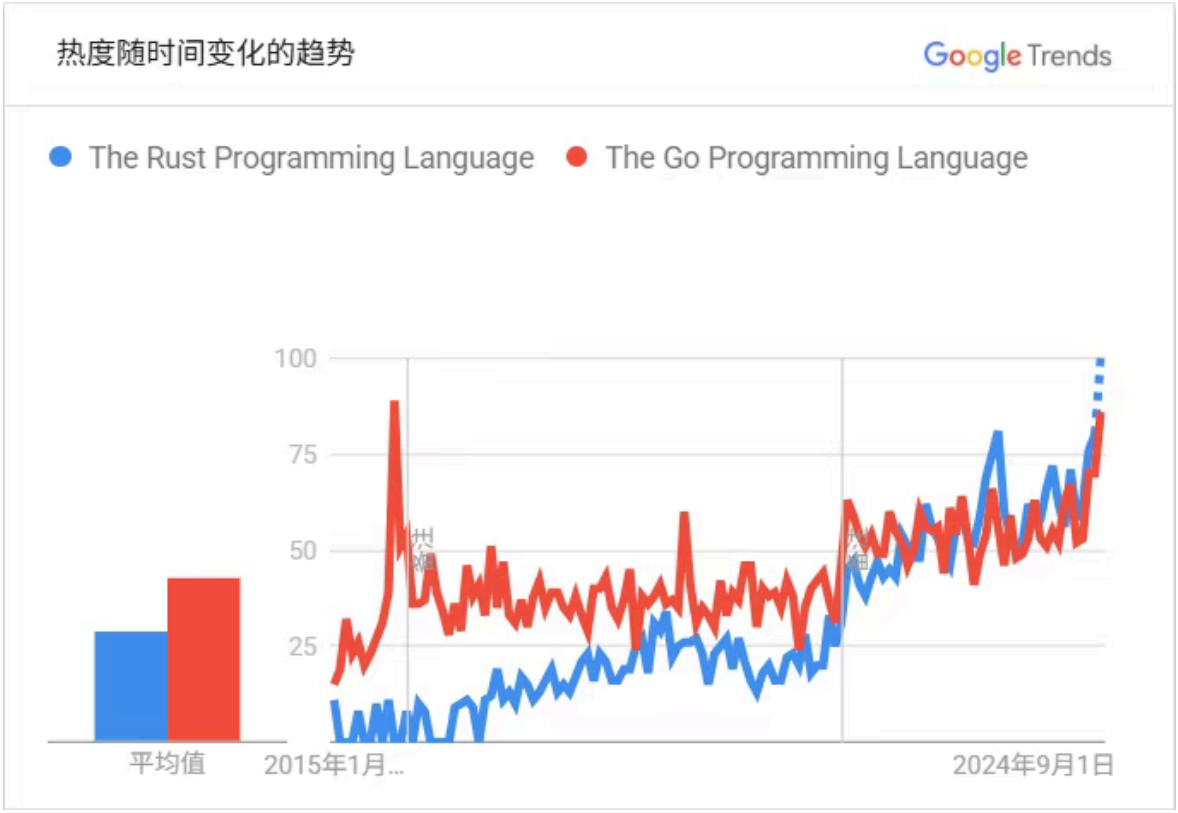
原创 腾讯程序员 腾讯技术工程 2025年10月13日 02:36 广东 1460人



作者：richardyao

最近几年，Rust在业界的使用越来越多，本篇文章从Rust核心语法等基础知识入手，进而详细的介绍Rust语言不同于其它语言的思维方式，最后通过一个实战的项目，带大家快速入门Rust编程语言。

最近几年，Rust在业界的使用越来越多。在Windows内核中(win32kbase_rs.sys)、Linux内核中、Chromium浏览器中都有Rust的身影，AWS S3也使用rust重构了他们的核心存储服务ShardStore，Azure的CTO甚至说"Speaking of languages, it's time to halt starting any new projects in C/C++ and use Rust for those scenarios where a non-GC language is required."。从下面的Google Trends也可以看的出来，Rust的热度正在上升，并且增长很快，可以说现在是学习Rust最好的时机。



本篇文章从Rust核心语法等基础知识入手，进而详细的介绍Rust语言不同于其它语言的思维方式，最后通过一个实战的项目，带大家快速入门Rust编程语言。

有C++、Golang、Python这些语言基础的话，大部分知识都可以迁移过去，再加上大模型的辅助，24小时快速入门，是有可能达成的。



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

690篇原创内容

公众号

一、基础篇

1.1 Rust的安装与基本工具的使用

Rust的安装直接参考官网的文档，这里不做更具体的介绍了：

https://www.rust-lang.org/tools/install

在国内使用Rust的话，可以通过 **https://rsproxy.cn/** 网站提供的镜像，更方便快捷的安装Rust以及下载相关的crate等。

Rust对应的编译器是rustc，但是这个大家平时使用的并不多，更多的是通过包管理工具cargo等来管理、构建项目。cargo常用命令如下：

```
cargo build
cargo build --release
cargo clippy
cargo run
cargo run -- --help
cargo clean
cargo check
cargo doc
cargo expand      # 需要使用cargo install cargo-expand先安装
```

Rust日常开发可以使用vscode + rust-analyzer插件。

1.2 通过greplite小程序熟悉Rust的语言特点

在这里，我们使用一个简单greplite程序来介绍Rust语言的特点。

先使用`cargo new greplite`命令创建一个binary的crate，然后在main.rs中输入下面的代码：

```
use std::env;
use std::fs::File;
```

```rust
use std::io::{self, BufRead, BufReader};

fn main() -> io::Result<()> {
    let args = env::args().collect::<Vec<_>>();
    if args.len() < 3 {
        eprintln!("Usage: greplite <search_string> <file_path>");
        std::process::exit(1);
    }

    let search_string = &args[1];
    let file_path = &args[2];

    run(search_string, file_path)
}

fn run(search_string: &str, file_path: &str) -> io::Result<()> {
    let file = File::open(file_path)?;
    let reader = BufReader::new(file);

    for line in reader.lines() {
        let line = line?;
        if line.contains(search_string) {
            println!("{line}");
        }
    }

    Ok(())
}
```

这个程序的第一个参数是要搜索的字符串，第二个参数是搜索的文件，比如说要搜索src/main.rs文件中包含main函数的行，可以如下执行：

```
$ cargo run -- main src/main.rs
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `target/debug/greplite main src/main.rs`
fn main() -> io::Result<()> {
```

这个小程序一共就30行代码，但是知识点还挺多的，先整体介绍一下：

a)、use类似于C++中的include，用来引用std标准库里面的module；

b)、fn用来定义一个函数，main函数是整个程序的入口；

c)、->表示函数的返回值，`io::Result<()>`表示返回的是个`Result`，通过文档，或者vscode代码跳转的方式，可以看到`std::io::Result<T>`是基于`std::result::Result<T, std::io::Error>`定义的一个新类型

https://doc.rust-lang.org/std/io/type.Result.html

```rust
pub type Result<T> = std::result::Result<T, std::io::Error>;
```

同时std::result::Result是一个枚举：

```rust
pub enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

Result表示的值，可以是表示成功的Ok，也可以是表示失败的Err。

上面pub type Result<T> = std::result::Result<T, std::io::Error>;只是把std::io::Result的错误类型固定成std::io::Error，表示只会返回std::io::Error类型的错误。

d)、io::Result<()>中的()，表示的是一个tuple，这个tuple没有任何元素，也称为unit；

e)、关于main函数的返回类型：

https://doc.rust-lang.org/reference/crates-and-source-files.html?highlight=main#main-functions

f)、let args = env::args().collect::<Vec<_>>();中的env::args()返回的值，实现了Iterator这个trait，关于Rust中的迭代器，后面还会重点来讲，这个collect是把迭代器的列表收集起来，构造成一个Vec。

这个语句还有下面几种写法，都是可以的：

```rust
let args = env::args().collect::<Vec<_>>();
let args = env::args().collect::<Vec<String>>();
let args: Vec<String> = env::args().collect();
let args: Vec<_> = env::args().collect();
```

Rust的类型推断是非常强大的，如果后面对args的使用能确定args的类型的话，也可以完全写成：

```rust
let args = env::args().collect();
```

比如说如果有函数fn print_args(args: &Vec<String>) {}，同时后面调用了print_args(&args)的话，args的定义就完全不需要类型注解了。

另外，通过let定义的变量，默认是不可变的(immutable)，如果要修改的话，需要显示的使用`let mut args = env::args().collect();`

这里还有一个知识点，我们在前面并没有通过use语句引入Vec，那为什么不报错呢?

https://doc.rust-lang.org/std/prelude/index.html

Rust编译器预先已经包含了std中部分常用的组件，这样代码会更简洁一些。

g)、`args.len()`中的len()是`Vec<String>`的成员函数 **https://doc.rust-lang.org/std/vec/struct.Vec.html#method.len**;

h)、`let search_string = &args[1];`和`let file_path = &args[2];`这里定义了两个引用，是对`Vec<String>`中对应元素的不可变的借用;

`args[1]`下标操作，在运行时会check下标是否越界，但是由于前面判断了长度，因此这里的运行时的越界检查，编译器通常会优化掉。

i)、下面是调用run函数，另外这里是一个expression，expression的值作为整个main函数的返回值;

j)、第19行`let file = File::open(file_path)?;`打开一个文件，这个语句中的?是一个语法糖，这条语句等价于:

```
    let file = match File::open(file_path) {
        Ok(f) => f,
        Err(err) => return Err(From::from(err)),
    };
```

https://doc.rust-lang.org/reference/expressions/operator-expr.html?highlight=question#the-question-mark-operator

h)、第20行`let reader = BufReader::new(file);`file实现了Read这个trait，然后BufReader是在Read这个trait的基础上，做了一层封装，实现了带缓存的读，并在此基础上，提供了lines()等便捷的方法。

https://doc.rust-lang.org/std/fs/struct.File.html#impl-Read-for-File

BufReader实现了BufRead这个trait，因此在`use std::io::BufRead;`之后，可以调用这个trait对应的lines()等方法。

https://doc.rust-lang.org/std/io/struct.BufReader.html#impl-BufRead-for-BufReader

i)、第22行`for line in reader.lines() {`lines()函数返回了一个迭代器，然后for line in的方式来遍历这个迭代器，这个迭代器对应的Item为

```
```

```
type Item = Result<String, Error>
```

https://doc.rust-lang.org/std/io/struct.Lines.html#associatedtype.Item

这个被称为trait的Associated type。

j)、这个迭代器的Item是一个Result，因此第23行使用?运算符把其转换成了普通的String;

k)、第24行`if line.contains(search_string) {`判断line是否包含要搜索的子串；

l)、第25行`println!("{line}");`打印输出，和`println!("{}", line);`等价，前面这种方式被称为named parameters：

https://doc.rust-lang.org/std/fmt/index.html#named-parameters

另外println!包括前面的eprintln!最后的这个!，表示这是一个宏。

Rust中，函数不支持可变参数，通过宏的方式来实现可变参数。

m)、最后第29行，Ok(())这个expression作为整个函数的返回值表示成功。

n)、同时我们注意到search_string和file_path都是&String类型的，run函数的参数&str是什么鬼？

类比于C++中的string和string_view，同时string到string_view可以通过string的operator string_view进行隐式转换：

https://en.cppreference.com/w/cpp/string/basic_string/operator_basic_string_view.html

在rust中，String到&str也可以进行隐式转换：

https://doc.rust-lang.org/std/string/struct.String.html#impl-Deref-for-String

https://doc.rust-lang.org/std/ops/trait.Deref.html#deref-coercion

> 思考题：1、在上面的30行代码中，一共涉及到哪些trait？Read、BufRead、Iterator、FromIterator、From、Deref、Termination、Drop2、在上面的30行代码中，一共有哪些迭代器iterator？std::env::Args、std::io::Lines

从上面的这个小例子中，我们也能一窥Rust程序的特点：

1、代码风格，下划线小写命名的形式；

2、倾向于使用trait，使用组合的方式来实现程序的功能；

3、迭代器iterator功能挺强大的；

4、Rust学习曲线确实很陡峭，30行代码竟然涉及这么多语法。

1.3 Rust中组织数据的3种方式

在Rust中，我们可以使用struct来组织数据。

```
struct Person {
    first_name: String,
    last_name: String,
    age: i32,
}
```

也可以使用tuple(Rust中的tuple和python中的tuple概念是一致的)：

```
let person_info = ("Harry", "Potter", 18);
let first_name = person_info.0;
let (first_name, _, _) = person_info;
```

Rust中的enum表示的可以是一个集合类型中的任意一种：

```
enum WebEvent {
    // An enum variant without any data.
    PageLoad,
    // An enum variant with a string slice.
    KeyPress(char),
    // An enum variant with a struct.
    Click { x: i32, y: i32 },
    // An enum variant with an owned String.
    Paste(String),
}

impl WebEvent {
    fn describe(&self) {
        matchself {
            WebEvent::PageLoad => println!("Page loaded"),
            WebEvent::KeyPress(c) => println!("Key pressed: {}", c),
            WebEvent::Click { x, y } => println!("Clicked at: ({}, {})", x, y),
            WebEvent::Paste(s) => println!("Pasted: {}", s),
        }
    }
}
```

enum通常配合match在一起使用。

1.4 Rust中的Ownership

Rust中的ownership规则：

```
1. Each value in Rust has an owner.
2. There can only be one owner at a time.
3. When the owner goes out of scope, the value will be dropped.
```

通过ownership的机制实现RAII，当变量离开作用域的时候，会被释放或者drop掉。

Rust在默认的情况下，是move语义的，比如说：

```
let a = vec![1, 2, 3];
let b = a;
println!("{:?}", a);
println!("{:?}", b);    // error[E0382]: borrow of moved value: `a`
```

但是如果对应的类型实现了Copy这个trait的话，默认就会走copy的语义：

```
let a = 1;
let b = a;
println!("{:?}", a);
println!("{:?}", b);
```

https://doc.rust-lang.org/std/marker/trait.Copy.html

Rust中为很多简单类型都自动实现了Copy这个trait。

Copy和Clone的区别：

```
#[derive(Copy, Clone)]
struct Point {
   x: i32,
   y: i32,
}
```

在上面Point的定义中，由于i32同时实现了Copy和Clone这两个trait，因此他们组合在一起，Point也能实现这两个Trait。

```
#[derive(Clone)]
struct PointList {
    points: Vec<Point>,
}
```

而PointList中，由于Vec只实现了Clone这两个trait，因此PointList也只能实现Clone这个trait，不能实现Copy。Copy和Clone的区别，就类似于浅拷贝和深拷贝的区别，上面Point的定义，两个变量都是分配在栈上的，浅拷贝和深拷贝没有区别，因此Point可以同时实现Copy和Clone这两个trait；而下面PointList的定义中，Vec在栈上只记录了元信息(pointer, capacity, length)，Vec的元素是存放在堆上的，只能深拷贝，因此只实现了Clone这个trait。

1.5 Rust中的引用和借用

```
1. At any given time, you can have either one mutable reference or any number of immut
2. References must always be valid.
```

由于Rust中默认是move语义的，在有的场景下，我并不想转移ownership，这种情况下，可以通过引用来借用。

引用分为两种，一种是immutable引用，一种是mutable的引用。

通过immutable的引用，借用者不能修改；通过mutable的引用，借用者可以对这个变量做任何的修改，比如说赋值、swap等，唯一的一个限制就是要保证这个变量的完整性。

Rust的安全机制要求引用在任何时候都必须有有效；同时，限制mutable引用和immutable引用不能同时存在：你可以有多个immutable的引用；也可以有一个mutable的引用；但是不允许有多个mutable的引用，也不允许mutable的引用和immutable的引用同时存在。

思考：为什么对同一个变量的mutable引用和immutable引用不能同时存在？思考下面的例子：

```
fn main() {
    let mut v = vec![1, 2, 3, 4];
    let first = &v[0];
    v.push(5);   // error[E0502]: cannot borrow `v` as mutable because it is also bor
    println!("first element: {}", first);
}
```

1.6 trait

Rust中的trait类似于golang中的interface，Rust中通过trait定义共同的行为。比如说咱们定义Shape形状这样的trait，所有的形状都有面积：

```
trait Shape {
    fn area(&self) -> f64;
}
```

圆和长方形都能实现Shape形状这个Trait：

```
```

```rust
struct Circle {
    radius: f64,
}

impl Shape for Circle {
    fn area(&self) -> f64 {
        std::f64::consts::PI * self.radius * self.radius
    }
}

struct Rectangle {
    width: f64,
    height: f64,
}

impl Shape for Rectangle {
    fn area(&self) -> f64 {
        self.width * self.height
    }
}
```

有了这些定义之后，就可以实现编译时的类型约束：

```rust
fn print_area1<S: Shape>(shape: &S) {
    println!("The area is: {}", shape.area());
}

fn print_area2(shape: &impl Shape) {
    println!("The area is: {}", shape.area());
}
```

上面这两种实现的方式是等价的，impl只是一个语法糖：

https://doc.rust-lang.org/stable/reference/types/impl-trait.html

也可以实现运行时的多态：

```rust
fn print_area3(shapes: &[&dyn Shape]) {
    for shape in shapes {
        println!("The area is: {}", shape.area());
    }
}
```

调用上面几个方法的例子：

```rust
```

```rust
fn main() {
    let circle = Circle { radius: 5.0 };
    let rectangle = Rectangle {
        width: 10.0,
        height: 4.0,
    };

    print_area1(&circle);
    print_area2(&rectangle);

    let shapes: Vec<&dyn Shape> = vec![&circle, &rectangle];
    print_area3(&shapes);
}
```

1.7 迭代器

Rust中的迭代器由Iterator这个trait来表示，表示会产生一个序列的值：

https://doc.rust-lang.org/std/iter/trait.Iterator.html

这个trait只有一个Required的方法next，当next返回None的时候，表示序列结束：

```rust
// Required method
fn next(&mut self) -> Option<Self::Item>;
```

可以直接调用next方法来一个一个的获取值，但是更多的场景下是使用标准库中提供的adapter和consume方法。

rust中的迭代器有以下特点：

a)、laziness

https://doc.rust-lang.org/std/iter/index.html#laziness

b)、infinity

https://doc.rust-lang.org/std/iter/index.html#infinity

c)、高效，性能很高，和手写for循环性能是一致的。

```rust
struct Fibonacci {
    current: u64,
    next: u64,
}

impl Fibonacci {
    fn new() -> Fibonacci {
        Fibonacci { current: 0, next: 1 }
```

```
    }
}

implIteratorfor Fibonacci {
    type Item = u64;

    fn next(&mutself) -> Option<Self::Item> {
        let next_number = self.current + self.next;
        self.current = self.next;
        self.next = next_number;
        Some(self.current)
    }
}

fn main() {
    let fib_iterator = Fibonacci::new();

    println!("The first 10 Fibonacci numbers are:");
    for number in fib_iterator.take(10) {
        println!("{}", number);
    }

    let fib_vec: Vec<u64> = Fibonacci::new().take(15).collect();
    println!("\nThe first 15 numbers collected into a vector:");
    println!("{:?}", fib_vec);
}
```

1.8 闭包

Rust中的闭包能够capture环境中的值，根据capture的方式不同，闭包也分别实现了不同的trait，如果是普通的borrow来capture的话，实现了Fn，如果是mut borrow来capture的话，实现了FnMut，如果是move consume了变量的话，实现FnOnce，看下面的例子：

```
fn main() {
    let s = String::from("hello");
    let f1 = || &s;
    println!("{}", f1());
    println!("{}", f1());
}
```

如上面的代码f1是一个闭包，capture了变量s的引用，编译器自动帮这个闭包实现了Fn的trait，这个闭包可以调用多次。我们也可以看到rust标准库中Fn这个trait的定义是 fn call(&self, args: Args) -> Self::Output;，传递的是self的引用，因此才可以调用多次。

```
fn main() {
    let mut s = String::from("hello");
```

```
    let mut f2 = || s += "world";
    f2();
    //println!("{}", s);
    f2();
    println!("{}", s);
}
```

如上面的代码，f2也是一个闭包，mut borrow了s，因此编译器自动帮这个闭包实现了FnMut这个trait，注意，上面的代码中，如果注释掉中间的println!的话，会报error[E0502]: cannot borrow s as immutable because it is also borrowed as mutable错误，f2是变量s的一个mut引用，要满足s引用的限制规则。FnMut在标准库中是这样定义的：fn call_mut(&mut self, args: Args) -> Self::Output;，可以看到第一个参数是&mut self。

```
fn main() {
    let s = String::from("hello");
    let f3 = || s;
    println!("{}", f3());
    //println!("{}", f3());  // error[E0382]: use of moved value: `f3`
}
```

上面的代码中f3成为了s的owner，实现了FnOnce这个trait，f3只能调用一次，第二次调用的话会报use of moved value的错误信息。FnOnce在标准库中的定义：fn call_once(self, args: Args) -> Self::Output;，self是move的这种调用方式，因此只能调用一次。

同时，编译器为实现了Fn的闭包，也同时实现了FnMut和FnOnce；实现了FnMut的闭包也同时实现了FnOnce。

https://doc.rust-lang.org/std/ops/trait.Fn.html

https://doc.rust-lang.org/std/ops/trait.FnMut.html

https://doc.rust-lang.org/std/ops/trait.FnOnce.html

1.9 Sync & Send

Rust中的并发安全，是通过Sync和Send这两个trait来体现的。Send表示的含义是，变量可以跨越线程的边界进行传递；Sync表示的含义是，变量可以多线程同时访问。

这里通过一个简单的例子，演示下Sync & Send如何保证并发安全的：

```
use std::sync::Arc;
use std::sync::Mutex;
use std::thread;
```

```rust
fn test1() {
    letmut a = vec![1, 2, 3];

    let handler = std::thread::spawn(move || {
        a.push(4);
    });

    handler.join().unwrap();
}

fn test2() {
    letmut a = vec![1, 2, 3];

    thread::scope(|s| {
        s.spawn(|| {
            println!("hello from the first scoped thread");

            a.push(4);
        });
    });

    a.push(5);
}

fn test3() {
    let a1 = Arc::new(vec![1, 2, 3]);
    let a2 = a1.clone();

    let handler = thread::spawn(move || {
        println!("a1 {:?}", a1);
    });

    println!("a2 {:?}", a2);

    handler.join().unwrap();
}

fn test4() {
    let a1 = Arc::new(Mutex::new(vec![1, 2, 3]));
    let a2 = a1.clone();
    let a3 = a1.clone();

    let handler1 = thread::spawn(move || {
        letmut lock_guard = a1.lock().unwrap();
        lock_guard.push(4);
    });

    let handler2 = thread::spawn(move || {
        letmut lock_guard = a2.lock().unwrap();
        lock_guard.push(4);
    });

    handler1.join().unwrap();
```

```
    handler2.join().unwrap();
    println!("a3 {:?}", a3.lock().unwrap());
}

fn main() {
    test1();
    test2();
    test3();
    test4();
}
```

1.10 async & await

Rust的异步编程，被称为无栈协程，先看一个简单的例子：

```
use anyhow::Result;
use serde::Deserialize;

#[derive(Deserialize, Debug)]
struct Joke {
    joke: String,
}

#[tokio::main]
asyncfn main() -> Result<()> {
    let res = reqwest::get("https://geek-jokes.sameerkumar.website/api?format=json").
    let joke = res.json::<Joke>().await?;
    println!("{}", joke.joke);
    Ok(())
}
```

使用cargo expand命令，上面的代码，main大概展开成下面的样子：

```
fn main() -> Result<()> {
    tokio::runtime::Builder::new_multi_thread()
        .enable_all()
        .build()
        .expect("Failed building the Runtime")
        .block_on(async {
            let res = reqwest::get("https://geek-jokes.sameerkumar.website/api?format=
            let joke = res.json::<Joke>().await?;
            println!("{}", joke.joke);
            Ok(())
        })
}
```

从上面的代码可以看到，代码中首先构建了一个tokio的runtime，然后block_on在某个

async块上进行执行。

async/await把Rust程序分割成了两个世界，在async/await的上下文里，不能调用阻塞的函数，不然会卡住异步运行时tokio的执行和调度。

为了弄清楚async/await到底干了啥，咱们首先看下上面代码中的其中一行代码：

```
let res = reqwest::get("https://geek-jokes.sameerkumar.website/api?format=json").awai
```

这行代码可以拆成两行：

```
let res_future = reqwest::get("https://geek-jokes.sameerkumar.website/api?format=json"
let res = res_future.await?;
```

在vscode里面，把鼠标悬停在res_future上，可以看到vscode给出的类型注解是：

```
let res_future: impl Future<Output = Result<Response, Error>>
```

可以看到res_future实现了Future这个trait，但是res_future具体的类型不知道，只知道他实现了Future这个trait。

async只是一个语法糖：

```
async fn test() {
    println!("This is a test function.");
}

fn test2() -> impl Future<Output = ()> {
    async {
        println!("This is a test function.");
    }
}
```

async的本质，实际上是编译器把reqwest::get("https://geek-jokes.sameerkumar.website/api?format=json")编译成了一个状态机，然后这个状态机实现了Future这个trait，所以这里get返回的时候，实际上并没有发出任何http请求，只是返回了一个状态机，这个状态机实现了Future这个trait，仅此而已。理论上来说，也可以手写一个struct，实现同样的状态机，只是这个过程会特别的复杂，编译器直接帮忙咱们做了：

https://doc.rust-lang.org/stable/reference/items/functions.html#r-items.fn.async

一些手动实现Future的例子：

https://docs.rs/futures-util/latest/futures_util/future/struct.Select.html#impl-Future-for-Select

https://docs.rs/tokio/latest/tokio/time/struct.Timeout.html#impl-Future-for-Timeout

await的本质，实际上是“不停的”调用上面状态机的poll方法，驱动状态机不停的往前走，直到Ready为止。

https://doc.rust-lang.org/stable/reference/expressions/await-expr.html?highlight=await#r-expr.await.effects

rust异步编程的核心，就是Future这个trait：

```
pub trait Future {
    type Output;

    // Required method
    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
}

pub enum Poll<T> {
    Ready(T),
    Pending,
}
```

上面Future这个trait的定义中，poll函数的第一个参数是个`Pin<&mut Self>`的类型，什么是Pin，为什么需要Pin呢?

还是要从上面async生成的状态机说起，`reqwest::get("https://geek-jokes.sameerkumar.website/api?format=json")`这个函数会返回一个对象，这个对象实现了Future trait，这个对象是一个状态机，内部维护这个请求的执行状态，然后await的时候，会“不停”的poll，驱动状态机往前走，这个状态机内部会维护很多的状态，比如说tcp socket收发的buffer，以及buffer已经使用的大小等。换句话说，这个状态机是一个自引用的对象，状态机内部有一些buffer，然后状态机内部有一些指针指向这些buffer的某些位置等。状态机是自引用的，就要求这个状态机不能在内存中随意的移动，如果移动的话，自引用指针的指向就错了，指向了别的位置。Rust对这个问题的解法就是加一层Pin，对这个状态机的所有的访问，都是通过Pin这个智能指针来访问的，Pin限制了这个状态机不能移动和复制。

异步任务的取消：不.await了，不poll了，异步的请求也就取消了，比如说前面提到的 **https://docs.rs/futures-util/latest/futures_util/future/struct.Select.html** 当其中的一个已经Ready之后，另外一个就自动的取消了。


腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

690篇原创内容

公众号

二、思维篇

每个语言都有自己的特点，比如说Golang推崇通过消息的方式共享内存，Python语言中的list comprehensions是一种强大且简洁的创建列表的方法等，这里介绍下Rust语言的思维方式。

2.1 expression

在Rust中，推崇简洁，表达式可以作为值进行赋值，或者作为返回值：

```
fn foo(x: usize) -> usize {
    x
}
```

表达式可以赋值：

```
let x = if 3 > 2 {
 1
} else {
 2
};
```

前面的greplite的main函数，也可以写成：

```
fn main() -> io::Result<()> {
    let mut args = env::args().skip(1);

    let (search_string, file_path) = match (args.next(), args.next(), args.next()) {
        (Some(s), Some(f), None) => (s, f),
        _ => {
            eprintln!("Usage: greplite <search_string> <file_path>");
            std::process::exit(1)
        }
    };

    run(&search_string, &file_path)
}
```

2.2 split

由于Rust中ownership及引用规则的限制，有些对象会有split的操作，比如说Vec，split成两个，每个只修改Vec的一部分元素，这样整体还是安全的。

再比如说socket，可以split成一个只收数据的socket，一个只发数据的socket，这样可以实现在一个线程中只收数据，在一个线程中只发数据。

https://docs.rs/tokio/latest/tokio/net/struct.TcpStream.html#method.into_split

2.3 无处不在的Option和Result

就像在golang中，if err != nil {}和if someData != nil {}无处不在一样，在Rust中，Option和Result也是随处可见；Option和Result都是enum，如果都用match来进行判断的话，代码的递进深度会比较深，另外代码也看起来会很冗余。

为此Rust为Option和Result提供了很多便利的操作。

a)、question mark operator

https://doc.rust-lang.org/reference/expressions/operator-expr.html#r-expr.try

b)、is_some()、is_none()、is_ok()、is_err()便捷函数

c)、let else赋值操作

```
let a = Some(1);
let Some(b) = a else {
  return;
};
```

d)、和迭代器的互操作等

https://doc.rust-lang.org/std/option/#method-overview

https://doc.rust-lang.org/std/result/#method-overview

熟练掌握上面的方法，会让代码更简洁。

2.4 match的不仅仅是enum

在Rust中，match通常用来作用在enum上，然后每个分支判断enum的每个变体。

但是match不仅仅可以用在enum上，在其它的场景中，match也能发挥大作用。

另外，match是exhaustive的，需要写出所有的可能的分支。

https://github.com/yaozongyou/rust-24-hour-crash-course/blob/bdf5bc30a67fe6b5649a8fff7cc25e2e0d19a0e6/mini-redis/src/connection.rs#L112

https://doc.rust-lang.org/rust-by-example/flow_control/match.html

2.5 宏强大的超乎想象

宏的3种场景：

a)、类似于println!

本质是编译器一些规则的替换

https://doc.rust-lang.org/stable/reference/macros-by-example.html?highlight=hygiene#hygiene

b)、类似于前面例子中的`#[tokio::main]`

本质是在编译期，把对应注解的函数的Token Stream给到这个宏，然后这个宏，在编译期生成新的代码。

一个简单这种宏的例子，可以参考下面的这篇文章：

乐学Rust：100行代码实现简易集成测试框架

c)、类似于Debug宏

https://doc.rust-lang.org/std/fmt/derive.Debug.html

本质是在编译期，把对应注解的对应的struct的Token Stream给到这个宏，然后这个宏，在编译期生成新的代码。

2.6 通过传递消息的方式共享内存

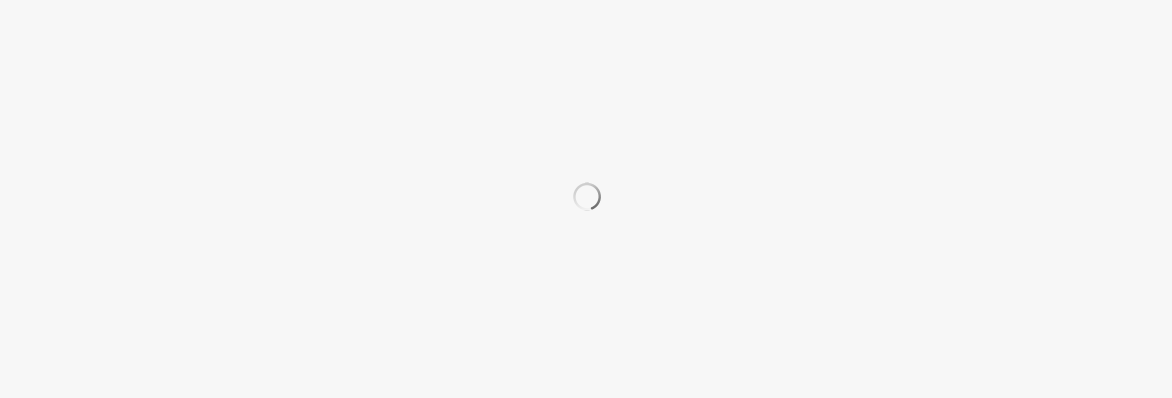
在Golang中，推崇通过消息的方式共享内存。同样，在Rust中，也支持这种编程的模式。

考虑对象从一个存储桶搬迁到另外的存储桶这种场景：

一种操作流程的组织形式可能类似于上图，左边的routine，调用list objects的接口，或者从文件列表中获取所有的对象列表，然后再把这些对象，放入到一个channel中；然后右边的copy的routine，具体执行copy每个对象的动作。

list操作和具体的copy操作在流程上做了分离，代码简洁清晰。

上面这个流程有一个小问题就是，当遇到大对象的时候，大对象可能会成为长尾的瓶颈，因为每个对象都是单routine拷贝的。流程上可以优化如下：



如上图，再加一层，真正的copy操作只在worker中进行处理，在copy这一层，把每个对象的copy任务，拆分成多个task，如果对象比较小的话，对应一个task，对象比较大的话，采用分块上传的方式，拆分成多个task。上图中的第二个channel里面的消息就是每个task任务，同时每个task任务中会包含另外一个channel，这个worker通过这个channel告知copy的routine对应的task的完成情况。

通知这种流程的组织形式，解决了大对象长尾的问题。流程依然清晰简洁。

在Rust中同样可以实现上面的这种流程模式。

不过对比Rust和Golang中channel和select的体验，由于Rust不是像golang那样，在语言本身支持channel和select，因此体感上，Rust稍微差了一丢丢。

2.7 和C++一致的内存模型

Rust采用和C++一致的内存模型，都是通过atomic原子操作来体现的，C++上的经验可以直接迁移到Rust上。

比如说，考虑配置热加载的场景，一种可能的实现是这样的，一个atomic的pointer指向当前的配置，然后有一个线程从本地或者通过sdk周期性从外部取最新的配置，然后再atomic的更新pointer指向最新的配置。

这里有个问题是，原来的配置何时释放的问题(safe memory reclamation)，在C++中，通常使用hazard pointer来解决，在Rust中也类似，也有hazard pointer。

不过最新的这种问题的解决方式，建议使用《Concurrent Deferred Reference Counting with Constant–Time Overhead》这篇paper介绍的方法，使用更方便：

https://github.com/cmuparlay/concurrent_deferred_rc

对应Rust的crate:

https://github.com/aarc-rs/aarc

关于Rust内存模型的书籍推荐。

https://marabos.nl/atomics/

2.8 Interior Mutability Pattern

由于Rust的ownership以及引用规则的限制，在写代码的时候，要想好各种数据结构，是否会跨多线程访问，如果跨多线程访问的话，可能要使用interior mutability pattern，所有的struct的函数都是&self，而不是&mut self。

参考例子：

https://github.com/yaozongyou/rust-24-hour-crash-course/blob/bdf5bc30a67fe6b5649a8fff7cc25e2e0d19a0e6/mini-redis/src/store.rs#L45

2.9 build.rs在编译期执行各种操作

crate有个build.rs脚本，可以获取代码仓库的git信息，编译c/c++程序等：

https://doc.rust-lang.org/cargo/reference/build-scripts.html

https://github.com/yaozongyou/rust-24-hour-crash-course/blob/bdf5bc30a67fe6b5649a8fff7cc25e2e0d19a0e6/mini-redis/build.rs

2.10 迭代器真的很好用

在Rust中，适当的使用迭代器会让代码更简洁。各种collection(Vec、HashMap、BTreeMap等)都能通过迭代器来遍历，Result和Option等也都能和迭代器相互转换等，迭代器也有特别多的adapter。

从C++转到Rust的话，可以尝试多使用下迭代器。

https://doc.rust-lang.org/std/iter/

之前greplite程序，可以改下成：

```
fn run(search_string: &str, file_path: &str) -> io::Result<()> {
    let file = File::open(file_path)?;
    let reader = BufReader::new(file);

    reader.lines().try_for_each(|line| {
        let line = line?;
        if line.contains(search_string) {
            println!("{}", line);
        }
```

```
        Ok::<(), _>(())
    })
}
```

或者

```
fn run(search_string: &str, file_path: &str) -> io::Result<()> {
    let file = File::open(file_path)?;
    let reader = BufReader::new(file);

    reader
        .lines()
        .collect::<io::Result<Vec<_>>>()?
        .into_iter()
        .filter(|line| line.contains(search_string))
        .for_each(|line| println!("{}", line));

    Ok(())
}
```

2.11 想定义个全局变量真不容易

关于全局变量，下面这篇文章总结的非常好:

https://www.sitepoint.com/rust-global-variables/

另外，上面这篇文章写的比较久了，上图中的lazy_static or once_cell，在当前最新的Rust的版本中，可以使用**OnceLock** 或者 **LazyLock**来替代，这样就不需要依赖第三方的crate了。

三、实战篇

使用Rust实现一个mini–redis:

https://github.com/yaozongyou/rust-24-hour-crash-course/tree/bdf5bc30a67fe6b5649a8fff7cc25e2e0d19a0e6/mini-redis

Rust学习建议

1、The Book通读一遍

2、**https://rustlings.rust-lang.org/**

rustlings上面的练习全部走一遍。

3、不要尝试写链表、不要尝试写链表、不要尝试写链表。

参考资料

1. **Using lightweight formal methods to validate a key-value storage node in Amazon S3**
2. **https://rsproxy.cn/**
3. **The Rust Programming Language**
4. **This Week in Rust**
5. **https://doc.rust-lang.org/nomicon/intro.html**
6. **https://doc.rust-lang.org/stable/reference/introduction.html**
7. **Rust Atomics and Locks**
8. **https://rustlings.rust-lang.org/**
9. **https://doc.rust-lang.org/rust-by-example/index.html**
10. **https://github.com/yaozongyou/rust-24-hour-crash-course**



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

690篇原创内容

公众号

阅读 3.3万